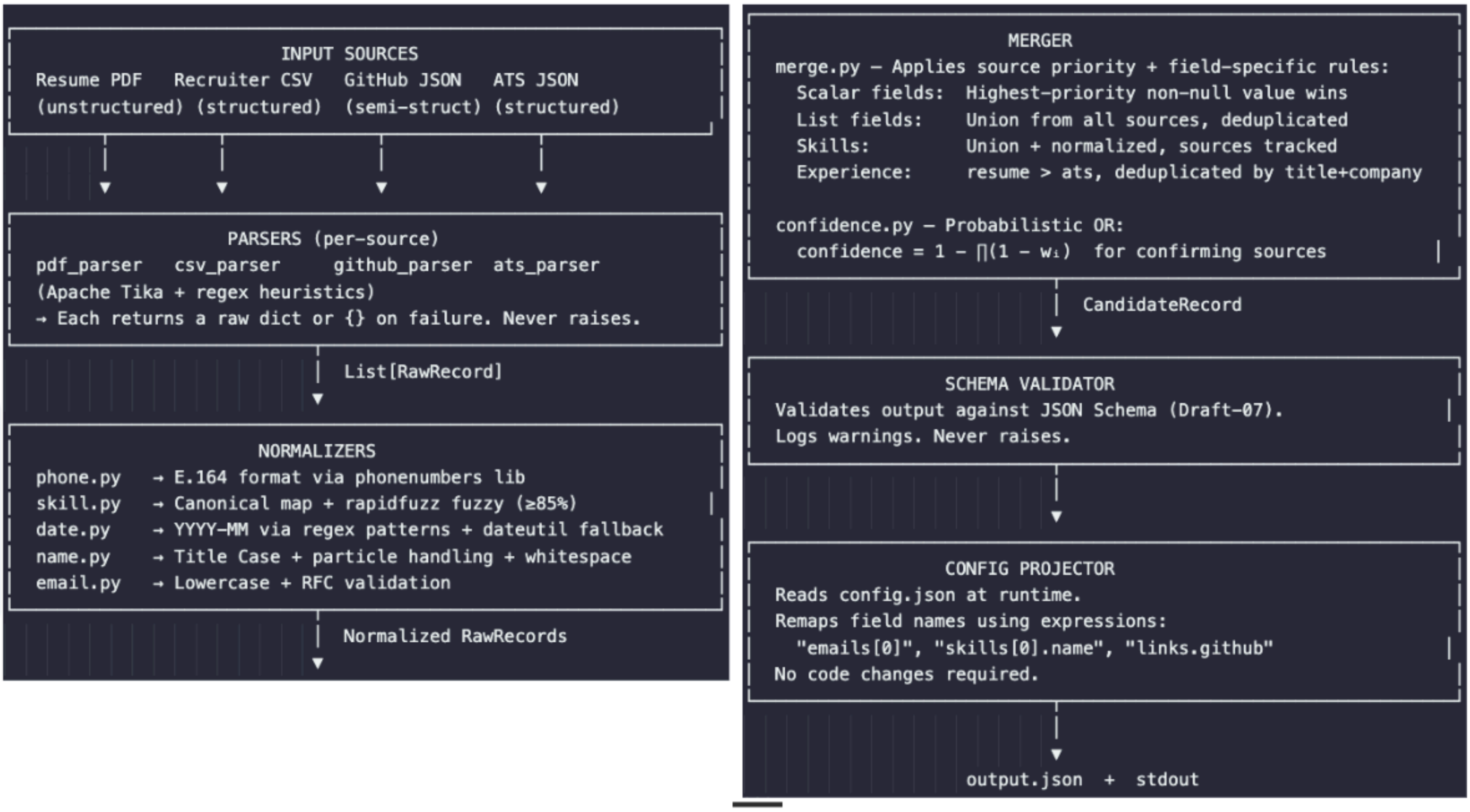
1. Problem Statement

Recruiters receive candidate information from many sources each with different schemas, field names, and formatting conventions. The goal is to build a pipeline that ingests multiple such sources, resolves conflicts, and produces a single, normalized, confidence-annotated JSON record.

2. Pipeline Diagram



3. Output Schema

{
"candidate_id": "string",
"full_name": "string | null",
"emails": ["string"],
"phones": ["string (E.164)"],
"location": "string | null",
"headline": "string | null",
"experience": [{ "title": "?", "company": "?", "start": "YYYY-MM", "end": "YYYY-MM | null", "is_current":true}],
"education": [{"degree": "?", "institution": "?", "year": "YYYY"}],
"skills": [{ "name": "?", "confidence": 0.0, "sources": ["..."]}],
"links": { "github": "url | null", "linkedin": "url | null" },

```
  "overall_confidence": 0.0,
  "sources_used": ["string"],
  "provenance": [{ "field": "?", "source": "?", "method": "?", "raw_value": "?" }]
}
```

4. Normalization Rules

~~~~
| Field | Input Examples | Output | Library |
|-------|---------------|--------|---------|
| Phone | `9876543210`, `+91 9876...`, `91-9876...` | `+919876543210` | `phonenumbers` |
| Skill | `nodejs`, `Node JS`, `NODE.JS` | `Node.js` | canonical map + `rapidfuzz` |
| Date  | `Jan 2023`, `01/2023`, `2023-01`, `2023` | `2023-01` | regex + `dateutil` |
| Name  | `JOHN DE SOUZA`, `john smith` | `John de Souza` | custom |
| Email | `ABC@Gmail.COM` | `abc@gmail.com` | `email-validator` |
~~~~

5. Merge Policy

~~~~
Source priority (highest → lowest):

| Rank | Source      | Rationale |
|------|-------------|-----------|
| 1    | Resume PDF  | Candidate-authored; most recently updated |
| 2    | GitHub JSON | Programmatically verified identity and skills |
| 3    | ATS JSON    | Recruiter-entered; may contain data-entry errors |
| 4    | CSV         | Bulk import with lowest editorial control |

Scalar conflict rule: For fields like `full_name`, `location`, `headline` — the value from the highest-ranked source that has a non-null entry wins.
Both the winning and losing values are recorded in `provenance`.

Field-specific overrides:
- `skills` — Union from all sources. GitHub gets confidence boost due to higher source weight.
- `emails / phones` — Union from all, normalized, deduplicated.
- `experience / education` — From resume > ats, deduplicated by key fields.
- `links` — Per-key first-available wins (github, linkedin separately).

Rationale: A resume is the primary artifact candidates prepare for job applications and is the most recently curated. GitHub confirms technical identity
but may lack professional narrative. ATS data is accurate at time of entry but may contain transcription errors. CSV is the most error-prone (bulk
operations, copy-paste).
~~~~

6. Confidence Policy

~~~~
Formula: Probabilistic OR (independent evidence model)

```
confidence = 1 − ∏(1 − w_i)
```

Each source is modeled as an independent "witness". The more witnesses agree on a fact, the higher the confidence — even if each witness alone is
imperfect.

Source weights:

| Source  | Weight | Basis |
|---------|--------|-------|
| GitHub  | 0.90   | Programmatically verified |
| Resume  | 0.85   | Candidate-authored |
| ATS     | 0.80   | Recruiter-entered |
| CSV     | 0.75   | Bulk import |
| Notes   | 0.50   | Informal / subjective |

Examples:
```
resume only:          1 − (1−0.85)          = 0.85
github only:          1 − (1−0.90)          = 0.90
resume + github:      1 − (0.15)(0.10)      = 0.985
resume + github + ats: 1 − (0.15)(0.10)(0.20)= 0.997 → capped 0.99
~~~~